Convolution as a local operation

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

Before you start

Why must an image schema record axis order as well as shape?

Plan time to read, investigate and explain. Keep predictions separate from observations.

Your mission

> Start with the target: Compute a local weighted sum at each valid image position. Try the recall before opening notes.

Your learning target

I can compute a local weighted sum at each valid image position.

I can explain shared weights and predict **output** dimensions.

I can check how **stride** boundaries and activation affect the result.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Image tensors and channels. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Debugging shapes and gradients. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** and validation curves. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

This week you will practise:

Compute a local weighted sum at each valid image position

Explain shared weights and predict **output** dimensions

Check how **stride** boundaries and activation affect the result

Retrieve what you already know

> Why must an image schema record axis order as well as shape?

My starting explanation

This week’s end goal

By the end, I can explain convolution as a local operation using a traced example and a stated limitation.

Notice the evidence

> Investigate the supplied Convolution as a local operation case. Separate observations, given facts and assumptions before choosing an explanation.

Use a fictional single-channel image with rows [1, 2, 0], [0, 1, 3] and [2, 0, 1]. Coordinates are zero-based row then column. Our **kernel** has rows [1, -1] and [0, 1].

At each position, multiply matching entries of a 2 by 2 window and the **kernel**, then add the four products. Use bias 0, **stride** 1 and no **padding**.

At the top-left window, the sum is 1 times 1 plus 2 times -1 plus 0 times 0 plus 1 times 1 = 0. Slide the same **kernel** right, then down. Only use windows that fit entirely inside the image; this is the valid boundary rule.

Window start	Four products	Sum
[0, 0]	1, -2, 0, 1	0
[0, 1]	2, 0, 0, 3	5
[1, 0]	0, -1, 0, 0	-1
[1, 1]	1, -3, 0, 1	-1

The **feature map** has rows [0, 5] and [-1, -1]. This neural-network convention keeps the **kernel** orientation fixed; mathematically it is cross-correlation. Mathematical convolution reverses the **kernel** along both spatial axes.

We use the fixed-orientation convention throughout, so every hand calculation and program agrees.

List the given facts and one fact that is not supplied.

See how it works

> Follow the mechanism for Convolution as a local operation. Connect each labelled input, deciding step and result.

The following function assumes nonempty rectangular numeric arrays and a positive integer **stride**. It rejects a **kernel** larger than the image.

It computes one **input** channel, one **output** map, zero bias and no **padding**; no activation is applied inside the function.

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
    return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
```

The **kernel** weights are reused at all four positions. There are four **kernel** parameters, not sixteen separate weights. If we add one shared bias for this **output** map, there are five parameters.

The number of multiply operations grows with the number of windows even though the parameter count stays fixed.

Connect the picture and the explanation

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

STUDENT LESSON · 4 OF 18

Compare the cases

> Compare the supplied cases for Convolution as a local operation. Identify the detail that changes the conclusion.

Compare a familiar case with a changed case

How many shared **kernel** weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change one thing

Change: Add shared bias2 after each patch sum.

Predicted result

Prediction: **Feature map** becomes [[-2,-2]].

Why the result follows

Reason: Bias adds2 to each **output**; it does not change patch products or **output** shape.

STUDENT LESSON · 5 OF 18

Words that unlock the idea

> Use the vocabulary of Convolution as a local operation to explain a concrete decision, rather than only memorising definitions.

Word	Meaning
Kernel	A small array of weights applied to a local input neighbourhood
Feature map	The spatial array of outputs produced by applying a filter
Stride	The distance between successive window starting positions
Padding	Values added around an input boundary under a declared rule

Use two terms to explain a step in this week's example.

Follow a worked example

Follow the worked trace for Convolution as a local operation, predict a step, then compare with its explanation.

One shared kernel produces four local sums

WINDOW START	KERNEL ROWS	OUTPUT VALUE
Row 0 column 0	[1 -1] and [0 1]	0
Row 0 column 1	[1 -1] and [0 1]	5
Row 1 column 0	[1 -1] and [0 1]	-1
Row 1 column 1	[1 -1] and [0 1]	-1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Change only input[0][0] from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Convolution as a local operation

Draw the exact patch-by-patch arithmetic of a local image operation.

The facts we will use

Supplied facts

Image rows [1, 2, 3],[4, 5, 6]; **kernel** rows [1, 0],[0,-1]. Use unflipped **kernel**, no **padding**, **stride**1, bias0.

Supplied facts

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

AI ACADEMY / AI-5/21

Image used by both local kernel windows

	col0	col1	col2
row0	1	2	3
row1	4	5	6

Read a cell using both its row and column labels.

Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-5/21

Convolution as a local operation

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. First patch

1 x 1+2 x 0+4 x 0+5 x (-1)=-4.



2. Second patch

2 x 1+3 x 0+5 x 0+6 x (-1)=-4.



3. Feature map

Output shape1 x 2 and map [[-4,-4]]; the same kernel is reused at each position.

CHANGE ONE THING

Add shared bias2 after each patch sum.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. First patch	1×1+2×0+4×0+5×(-1)=-4.
2. Second patch	2×1+3×0+5×0+6×(-1)=-4.
3. Feature map	Output shape1×2 and map [[-4,-4]]; the same kernel is reused at each position.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Why the result follows

A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

What this example does not establish

Keep the stated limit

A local filter detects a numerical pattern; one **feature map** does not establish recognition or learned visual understanding.

Keep the stated limit

Read the labelled picture: Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Keep the stated limit

Change one thing: Add shared bias2 after each patch sum. Predict the result before checking the explanation. What remains the same?

STUDENT LESSON · 7 OF 18

Find and repair a mistake

> Find the unsupported step in the Convolution as a local operation claim and explain a repair using the supplied evidence.

Check the claim against the facts

For no **padding** and a **kernel** that fits, **output** height is floor((H - Kh) / s) + 1 and width is floor((W - Kw) / s) + 1, where s is the positive integer **stride**. Floor means round down.

Count allowed start positions before using the formula. With a 3 by 3 image, 2 by 2 **kernel** and **stride** 2, the only row and column start is 0, so the **output** is 1 by 1.

Check the claim against the facts

Padding changes which windows exist and what they contain. Adding one border of zeros on every side changes this 3 by 3 **input** to 5 by 5. At **stride** 1 with our 2 by 2 **kernel**, **output** shape becomes 4 by 4.

Its top-left window is [[0, 0], [0, 1]] and its **output** is 1. Zero **padding** is a chosen boundary assumption, not a measured black region outside the picture.

If ReLU is applied after the unpadded sums, the **feature map** becomes [[0, 5], [0, 0]]. Negative responses are valid weighted sums before activation; they are not necessarily a software error.

Keep raw sums available when debugging so ReLU does not hide a mistake.

Locate the earliest unsupported step

For the lesson image and **kernel**, give the **output** shape with no **padding** and **stride** 2. Then give the shape with one zero border on each side and **stride** 1, and calculate that padded map's top-left **output**.

State one repair and the checks you would rerun.

STUDENT LESSON · 8 OF 18

Practise together

Practise Convolution as a local operation with support, then explain the deciding step in your own words.

Use the lesson image and **kernel** at window start [0, 1]. List the window rows, four products and **output** before activation.

Change only input[0][0] from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

Explain your prediction to a partner before comparing results.

Practise with less help: Separate parameters and operations

Use workbook W2 for the connected example “Convolution as a local operation”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Convolution as a local operation. Proposed explanation: “Each window has its own set of learned weights”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

STUDENT LESSON · 9 OF 18

Run the investigation

> Investigate Convolution as a local operation. Record the prediction, actual observation or labelled paper trace, explanation and next test separately.

Predict, investigate and preserve the evidence

> How many shared **kernel** weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
    return out
```

```
x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
```

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

STUDENT LESSON · 10 OF 18

Build your understanding

> Move from recall to tracing, diagnosis and transfer in Convolution as a local operation. Explain why each step is justified.

Explain → apply → challenge

Use the lesson image and **kernel** at window start [0, 1]. List the window rows, four products and **output** before activation.

For the lesson image and **kernel**, give the **output** shape with no **padding** and **stride** 2. Then give the shape with one zero border on each side and **stride** 1, and calculate that padded map's top-left **output**.

A program reverses both axes of the lesson **kernel** to [[1, 0], [-1, 1]]. Compute the top-left **output**. Does it agree with our declared convention? Explain what the test checks.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

STUDENT LESSON · 11 OF 18

Connect your project

> Apply this week's idea to Image-pattern Network Investigation. Keep an annotated decision and the evidence supporting it.

Add a labelled convolution as a local operation **evidence** sheet to Image-pattern Network Investigation. Show how you can compute a local weighted sum at each valid image position.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	

Project evidence	Record
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Evidence for your project

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

STUDENT LESSON · 12 OF 18

Show what you know

> Use a genuinely new case for independent evidence. Previously discussed examples remain practice; use the separately named independent case before feedback.

These **kernel** weights were chosen for arithmetic practice, not fitted from data. A response of 5 is neither a probability nor proof that an object was recognised.

A trainable convolutional layer adjusts shared parameters using **training** examples. Additional channels require summing contributions across channels; that is beyond this single-channel calculation.

A teammate reports a 3 by 3 output for the unpadded lesson image and calls its largest number a probability. Identify the missing or incorrect assumptions and give the expected shape and meaning under our declared setup.

Sharing weights applies the same local rule at each allowed position. It does not guarantee that a whole image classifier ignores object location, especially when **stride**s, boundaries or later layers change the result.

Next week studies pooling and the spatial information it can discard.

A program reverses both axes of the lesson **kernel** to [[1, 0], [-1, 1]]. Compute the top-left **output**. Does it agree with our declared convention? Explain what the test checks.

My limited claim and a useful next check

STUDENT LESSON · 13 OF 18

Study a complete case

> Revisit the complete Convolution as a local operation case for supported practice. Repeated examples do not become fresh assessments.

Complete evidence record

One shared kernel produces four local sums

WINDOW START	KERNEL ROWS	OUTPUT VALUE
Row 0 column 0	[1 -1] and [0 1]	0
Row 0 column 1	[1 -1] and [0 1]	5
Row 1 column 0	[1 -1] and [0 1]	-1
Row 1 column 1	[1 -1] and [0 1]	-1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

Try a new case independently

> Attempt the reserved new case before feedback. Record support; the earlier examples below remain additional practice.

Independent case: Filter a new image independently

Use image rows [2, 1, 0], [1, 3, 2] and **kernel** rows [0, 1], [-1, 2]. Use no **padding**, **stride** 1 and bias 0. Give the **output** shape, both four-product sums and the **feature map**. Then add a shared bias of -5 and apply ReLU.

> **My independent explanation, trace and limits**

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Use image rows [2, 1, 0], [1, 3, 2] and **kernel** rows [0, 1], [-1, 2]. Use no **padding**, **stride** 1 and bias 0. Give the **output** shape, both four-product sums and the **feature map**. Then add a shared bias of -5 and apply ReLU.

> **My independent procedure, result and limitation**

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

STUDENT LESSON · 15 OF 18

Your lesson route

> Use this map to navigate Convolution as a local operation, connect the goals to tasks and keep one evidence record for each attempt.

Start with this question: Draw the exact patch-by-patch arithmetic of a local image operation.

By the end, I can	How I will show it
Compute a local weighted sum at each valid image position	Explain the deciding step, show a trace or test, and state one limit.
Explain shared weights and predict output dimensions	Explain the deciding step, show a trace or test, and state one limit.
Check how stride boundaries and activation affect the result	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

STUDENT LESSON · 16 OF 18

Finish and return

> Explain Convolution as a local operation without notes, identify a remaining question and plan a later check with a changed case.

Close your notes. Explain this goal in your own words: Compute a local weighted sum at each valid image position

> **My deciding step, evidence and one remaining question**

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

STUDENT LESSON · 17 OF 18

Supported practice, repair and portfolio

> Attempt supported questions before using hints or feedback. Review a missing building block, then keep the completed work.

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Draw the exact patch-by-patch arithmetic of a local image operation.

Practice 1: explain the deciding evidence

Which labelled step matches this detail? **Output** shape1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.

1. **Feature map**

2. First patch

3. Second patch

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Feedback · compare after trying

After your attempt, compare with the supported explanation: **Feature map Output** shape1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.

A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

Practice 2: explain the deciding evidence

Changed case · predict first

Changed-case practice: Add shared bias2 after each patch sum. Which conclusion is supported by the supplied facts?

1. We can decide the result without examining the changed **input** or the procedure.

2. **Feature map** becomes [[-2,-2]].

3. This one example guarantees correct results for every future **input**.

My selected conclusion, deciding fact and explanation

Hint · try first

Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Feedback · compare after trying

After your attempt, compare with the supported explanation: **Feature map** becomes [[-2,-2]]. Bias adds2 to each **output**; it does not change patch products or **output** shape.

Repair a missing building block

Review Image tensors and channels (ai-5/20). Goal: Read a pixel using declared row column and channel axes. Short practice: Where is the bottom green pixel in the first batched image?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Debugging shapes and gradients (ai-5/18). Goal: Check dimension contracts before computing a layer. Short practice: What should a debugging record pair with every gradient value?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** and validation curves (ai-5/15). Goal: Compute comparable **training** and validation losses at saved checkpoints. Short practice: Can the original epoch 3 **training** loss 1 justify claiming best generalisation?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Predict, run and debug

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Keep your completed work

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

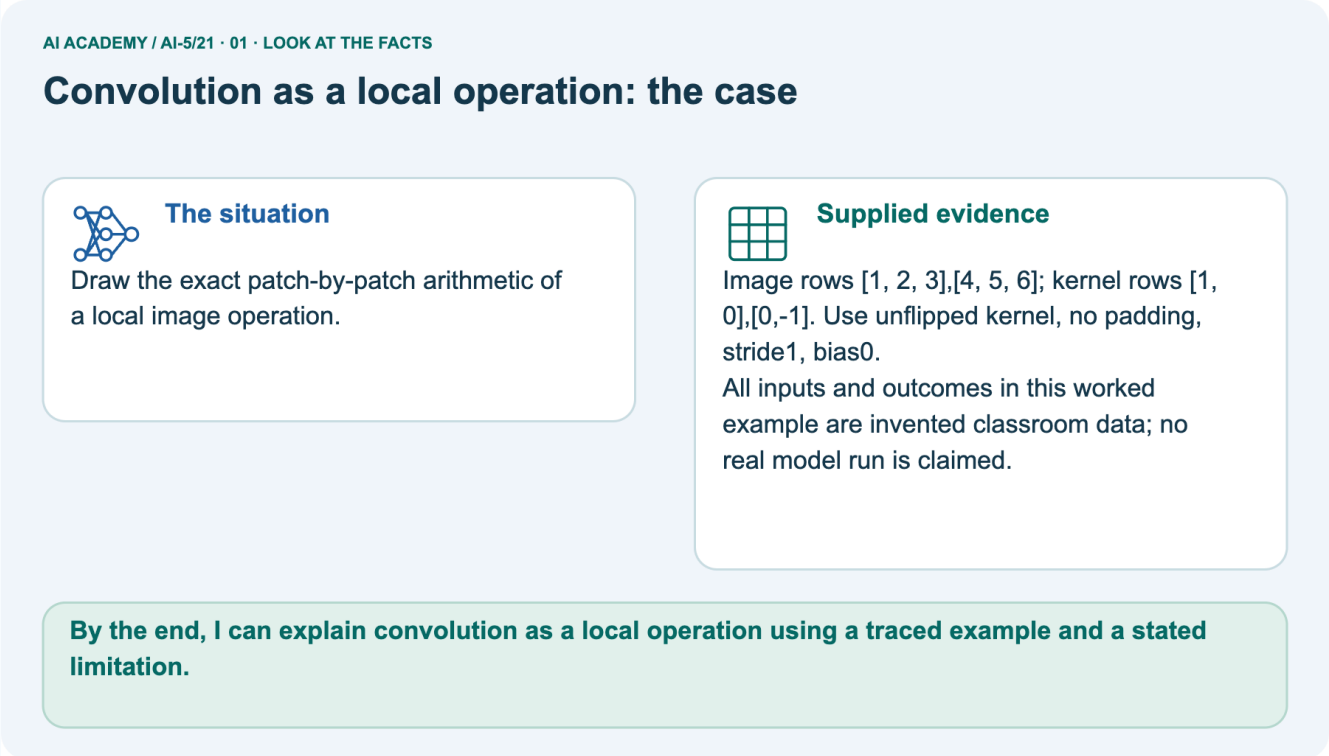
Drafts stay in this browser when storage is available; clear drafts on a shared device.

STUDENT LESSON · 18 OF 18

Visual concept journey

> Explain each Convolution as a local operation graphic using its labels, deciding evidence and relevant limitation.

01. Convolution as a local operation: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

Draw the exact patch-by-patch arithmetic of a local image operation. Supplied facts: Image rows [1, 2, 3],[4, 5, 6]; **kernel** rows [1, 0],[0,-1]. Use unflipped **kernel**, no **padding**, **stride**1, bias0.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

First patch: 1×1+2×0+4×0+5×(-1)=-4. → Second patch: 2×1+3×0+5×0+6×(-1)=-4. → **Feature map**: **Output** shape1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.

Point to each numbered stage and explain which recorded **input** it uses.

03. Image used by both local kernel windows

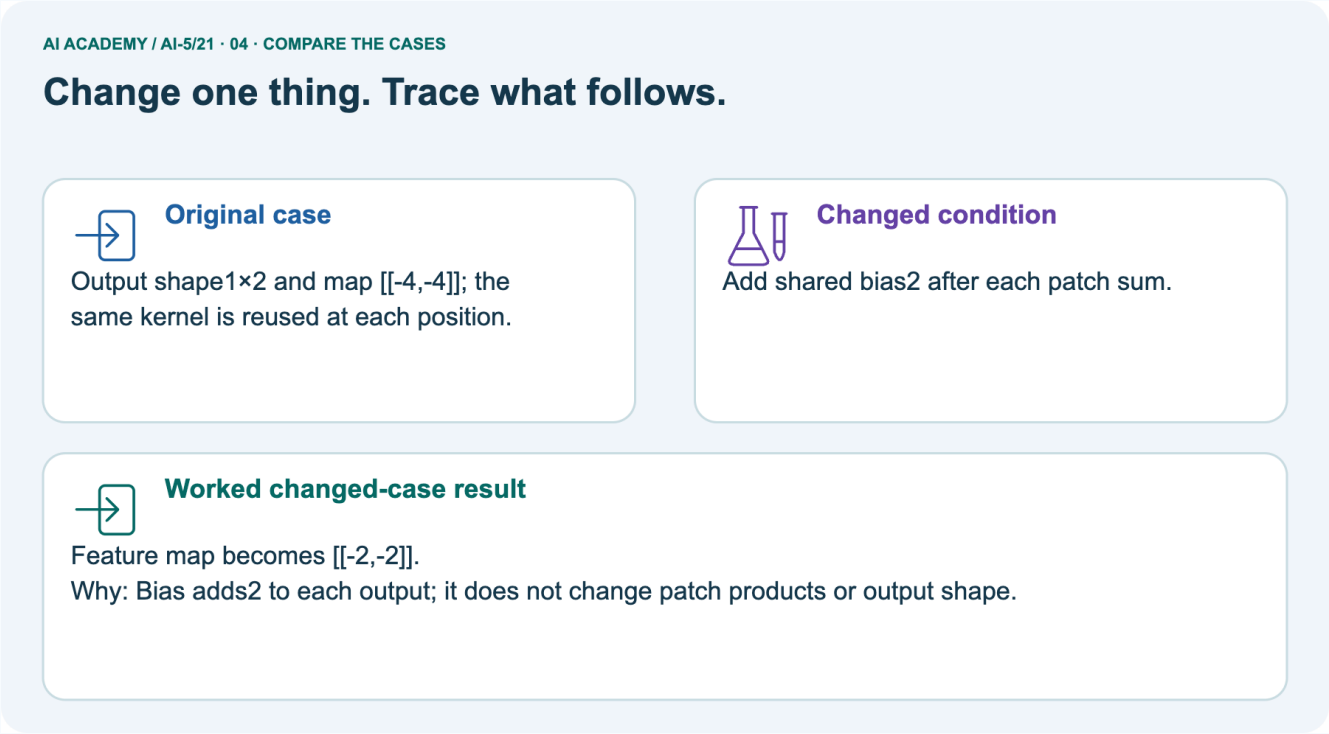


Follow the labelled picture with the matching explanation. Open the vector diagram

Image used by both local **kernel** windows. row0: col0 = 1, col1 = 2, col2 = 3. row1: col0 = 4, col1 = 5, col2 = 6.

> Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



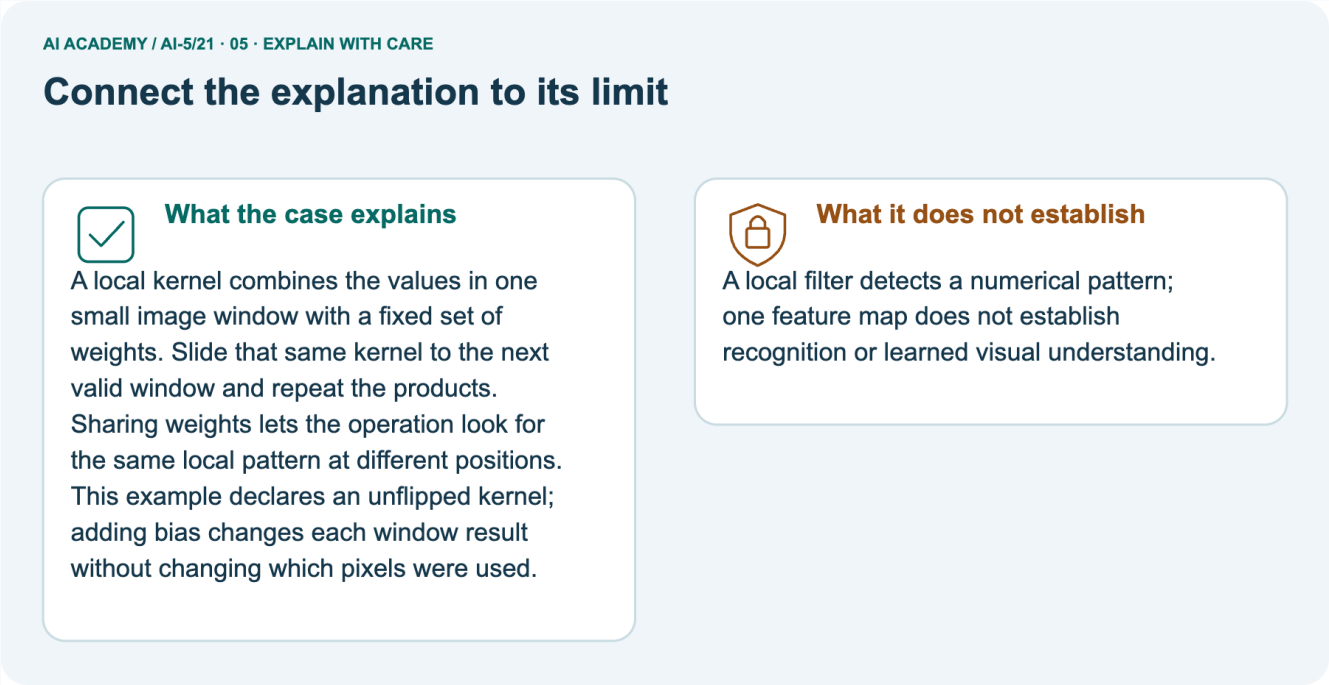
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: **Output** shape1×2 and map [[-4,-4]]; the same **kernel** is reused at each position. Changed condition: Add shared bias2 after each patch sum. Expected result: **Feature map** becomes [[-2,-2]].

Reason: Bias adds2 to each **output**; it does not change patch products or **output** shape.

> Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

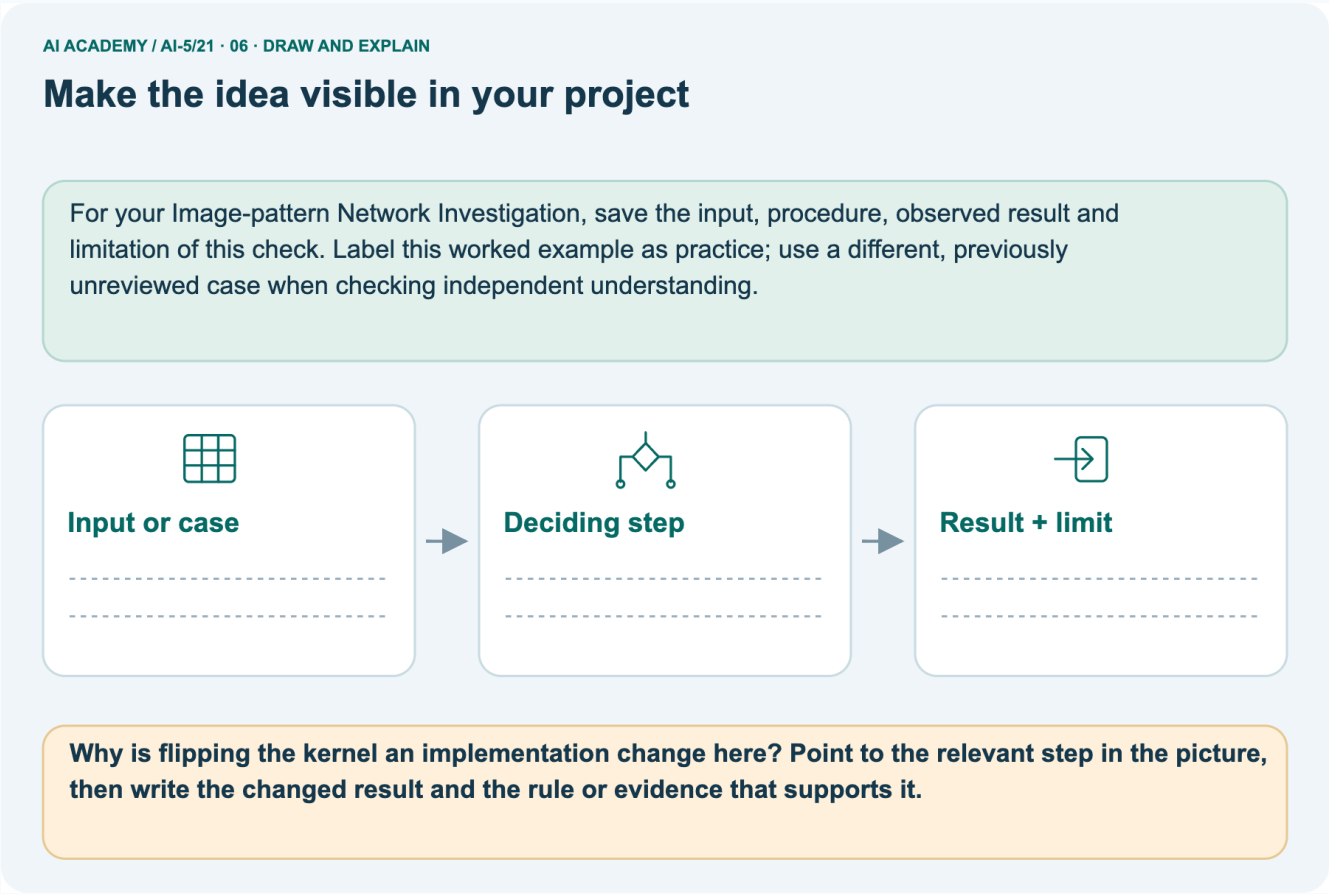
A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

Limit: A local filter detects a numerical pattern; one **feature map** does not establish recognition or learned visual understanding.

> Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> Draw your project trace and justify the deciding step using **evidence**.

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.